


VERBALE ESTERNO

# Incontro conoscitivo con VIMAR

22 ottobre 2026

| | |
|---|---|
| IDENTIFICATIVO | VE 2026-10-22 |
| VERSIONE | 1.0.2 |
| REDATTORE | Francesco Mazzurana |
| VERIFICATORE | Luca Cesco Bolla |
| DESTINATARI | Prof. Tullio Vardanega<br>Prof. Riccardo Cardin<br>VIMAR |

**Git Push Team** · Gruppo 9
gitpushteam@gmail.com

Università degli Studi di Padova
Ingegneria del Software · A.A. 2026/2027

## Registro delle modifiche

| VERSIONE | DATA | AUTORE | VERIFICATORE | DESCRIZIONE |
|---|---|---|---|---|
| 1.0.2 | 2026-10-26 | Francesco Mazzurana | Luca Cesco Bolla | Verifica e approvazione |
| 1.0.1 | 2026-10-26 | Francesco Mazzurana | – | Applicate correzioni a seguito della revisione del proponente |
| 1.0.0 | 2026-10-22 | Francesco Mazzurana | Luca Cesco Bolla | Verifica e approvazione |
| 0.1.0 | 2026-10-22 | Francesco Mazzurana | Luca Cesco Bolla | Prima stesura |

## Informazioni sulla riunione

| | | | |
|---|---|---|---|
| DATA | 22 ottobre 2026 | ORARIO | 16:00 – 17:00 |
| LUOGO | Microsoft Teams | TIPO | Verbale esterno |
| REDATTORE | Francesco Mazzurana | VERIFICATORE | Luca Cesco Bolla |

### Partecipanti del gruppo

| MEMBRO | MATRICOLA | PRESENZA |
|---|---|---|
| Enrico Biasini | 2147948 | • Presente |
| Luca Cesco Bolla | 2138006 | • Presente |
| Alessio Corrò | 2145607 | • Presente |
| Francesco Guglielmo | 2137975 | • Presente |
| Francesco Mazzurana | 2149463 | • Presente |
| Gaetano Spagnoli | 2147977 | • Presente |

### Partecipanti esterni

| NOME | RUOLO E ORGANIZZAZIONE |
|---|---|
| Mariano Sciacco | Referente, VIMAR |
| Domenico Bevilacqua | Referente, VIMAR |

# 1 Ordine del giorno

1. Risposte del proponente ai temi emersi dal sondaggio tra i gruppi.
2. Domande libere (Q&A dal vivo).
3. Prossimi passi: redazione e firma del verbale esterno.

# 2 Svolgimento

L'incontro è stato organizzato da Vimar S.p.A. prima dell'assegnazione dei capitolati ed era aperto a tutti i gruppi interessati al capitolato C1 – AURA. Prima dell'incontro i gruppi avevano inviato le proprie domande tramite un sondaggio: otto gruppi hanno posto 37 domande, che il proponente ha raggruppato in 14 temi e quattro aree (architettura e linguaggi, tecnologie e sicurezza, sviluppo e test, organizzazione).

Il Git Push Team aveva posto domande sui temi dei comandi critici e degli argomenti da studiare subito (temi 6 e 13). Le risposte a tutti i temi sono riportate nella sezione seguente.

# 3 Domande e risposte

## 3.1 Architettura e linguaggi

**D** **Quale linguaggio e quale SDK vanno usati per il server MCP?**

**R** Il linguaggio è libero (Python, Java o TypeScript): il proponente non pone vincoli. È consigliato Python con FastMCP, l'SDK suggerito nel capitolato e quello su cui Vimar può offrire il supporto più diretto. Usare lo stesso SDK per client e server semplifica i test di integrazione.

**D** **Che ruolo deve avere il client MCP?**

**R** Lo scopo finale è che un LLM possa usare il server MCP del gruppo; i client commerciali (Claude, ChatGPT...) includono già un client MCP standard. Un client proprio serve soprattutto per i test di integrazione; CLI, TUI ed estensione per VS Code sono opzionali. La divisione della gestione degli errori tra client e server (es. dispositivo non raggiungibile) va decisa in progettazione. Il punto principale resta il controllo in lettura e scrittura dei dispositivi.

**D** **Cosa viene fornito all'utente finale per collegare il proprio LLM?**

**R** Non serve sviluppare un'app né un sito: la soluzione è principalmente back-end e l'utente usa l'interfaccia dei servizi LLM. Si consiglia un server MCP locale per sviluppo e debug e, per l'utente, un server MCP remoto con URL pubblico e autenticazione, usabile da web, desktop e mobile. Basta che funzioni con un servizio AI commerciale; supportarne più di uno è un valore aggiunto, senza interfacce di selezione.

**D** **Quali risorse LLM e cloud saranno disponibili e cosa serve per la consegna?**

**R** Si possono usare i piani gratuiti o per studenti dei servizi commerciali (es. connettore personalizzato di Claude, Copilot con il GitHub Student Pack). Su richiesta, da discutere all'avvio, Vimar può dare accesso a un insieme ristretto di servizi AWS e a una chiave LLM per

le prove. Per la consegna basta che la soluzione giri in locale con Docker: al collaudo Vimar la installa su AWS o su un proprio server. Con il GitHub Student Pack sono disponibili 100$ di Microsoft Azure, in alternativa al piano gratuito di AWS.

## 3.2 Tecnologie e sicurezza

**D** **Come funziona l'autenticazione MyVimar in chat?**

**R** Serve un account MyVimar con accesso all'impianto; ogni utente vede solo i propri impianti, anche più di uno. Il server MCP associa ogni utente alla sua autorizzazione OAuth2 e la usa verso le API: il servizio AI non vede mai i token. Il flusso di login (es. link di autorizzazione da aprire nel browser, permessi da concedere) va progettato dal gruppo. I dati dell'impianto possono arrivare all'LLM per elaborare la risposta, i dati sensibili mai.

**D** **Comandi critici: chi blocca le richieste sbagliate?**

**R** Le API Vimar non hanno protezioni specifiche per l'uso con un LLM: leggono e scrivono come qualsiasi app. Le letture possono essere dirette; per le scritture (es. "spegni tutto") si può prevedere una conferma dell'utente. Gli *scope* KNX IoT permettono permessi configurabili (sola lettura, lettura e scrittura, solo alcuni dispositivi). Guardrail o skill aggiuntive sono ammessi se motivati nei requisiti.

**D** **Per lo stato e le notifiche serve il polling o la subscription?**

**R** Per questo progetto il polling va bene, come previsto dal capitolato. In genere un servizio AI non riceve messaggi avviati da un webhook, ma si può esplorare se qualche servizio lo consente. Il caso #21 ("avvisami quando…") è un nice-to-have realizzabile come azione programmata; le subscription vanno valutate solo dopo aver completato i casi base.

**D** **Quali richieste deve saper gestire l'assistente?**

**R** Non solo le frasi degli esempi: l'LLM deve gestire varianti, sinonimi e richieste ambigue. I casi base e avanzati del capitolato sono un punto di partenza per i requisiti, non un elenco esaustivo; nuove interazioni sono un nice-to-have gradito. Per aumentare la variabilità si consiglia di far riformulare a un LLM le domande della test list e di provarle con modelli diversi.

## 3.3 Sviluppo e test

**D** **Come si usa e si condivide il kit di impianto?**

**R** Il kit si collega al Wi-Fi e si comanda via cloud con le API KNX IoT, senza USB né VPN. Un membro del gruppo lo tiene a casa, sempre acceso e collegato, e gli altri lavorano da remoto tramite le API. Vimar fornisce un account amministratore a cui aggiungere altri utenti registrati su MyVimar.

**D** **Esiste un ambiente simulato per sviluppare in parallelo?**

**R** Dalla specifica OpenAPI di KNX IoT si può generare in locale un mock server (es. con Prism); esiste anche una demo online con API di mocking, inclusa nella documentazione per integratori. Il mock non replica esattamente il concetto di impianto di Vimar: la validazione finale va fatta sul kit. Il mock serve per lo sviluppo quotidiano, il kit per i test end-to-end.

**D** **Cosa misurano i test end-to-end e come si gestisce la CI?**

**R** La copertura dell'80% dei test end-to-end si misura sui requisiti di progetto; la test list del capitolato copre i casi base. Nei test di unità e di integrazione si può simulare il servizio AI, così i test sono ripetibili e senza costi; per questi vale la copertura del codice, almeno 75% e facoltativamente oltre il 90%. Un LLM reale non è deterministico, quindi è meglio tenerlo fuori dalla CI.

### 3.4 Organizzazione

**D** **Quali sono le priorità e cosa ci si aspetta dal PoC?**

**R** Il capitolato descrive esigenze di business: requisiti formali e priorità li definisce il gruppo nella propria analisi. Il PoC minimo è un server MCP funzionante su due o tre casi base: accesso all'impianto, elenco degli impianti associati, stato di una luce. Né gli opzionali né i nice-to-have sono obbligatori; i nice-to-have sono preferibili.

**D** **Cosa conviene studiare subito e quali difficoltà hanno avuto i gruppi degli anni scorsi?**

**R** Non c'è una scadenza: il lavoro parte dopo l'assegnazione, ma esplorare ora le tecnologie aiuta a scegliere. Conviene partire dalla documentazione KNX IoT per integratori, dalle primitive Vimar rispetto al protocollo e dallo standard MCP. Le difficoltà principali degli anni scorsi sono state requisiti poco chiari all'inizio, documentazione migliorabile e i limiti degli LLM eseguiti in locale: meglio chiarire presto i dubbi con Vimar e usare i servizi commerciali.

**D** **Come si svolgono SAL e incontri durante il progetto?**

**R** I SAL si tengono di solito su Teams, con la possibilità di incontri in sede Vimar. Sono previsti due incontri obbligatori in presenza (consegna del kit e collaudo finale) e indicativamente due demo con i colleghi di R&D, al PoC e all'MVP. Vimar è disponibile per dubbi progettuali e scelte architetturali, ma non può revisionare i documenti per la parte accademica. Approfondimenti mirati vanno richiesti con una o due settimane di anticipo.

### 3.5 Domande libere

Nella seconda parte dell'incontro i gruppi hanno potuto porre domande libere tramite il Q&A di Teams o intervenendo a voce.

## 4 Prossimi passi

Il gruppo procede alla redazione del verbale esterno e all'invio da VIMAR per la firma digitale.

## 5 Decisioni

| ID | DECISIONE |
|---|---|
| D1 | Il gruppo redige il verbale esterno dell'incontro e lo invia ai referenti Vimar per l'approvazione con firma digitale. |
| D2 | Le risposte ricevute vengono usate nella valutazione dei capitolati in vista della scelta. |

## 6 Azioni da intraprendere

| ID | ATTIVITÀ | RESPONSABILE | SCADENZA | ISSUE |
|---|---|---|---|---|
| A1 | Inviare il verbale ai referenti Vimar per la firma digitale. | Francesco Mazzurana | 2026/10/22 | – |
| A2 | Approfondire la documentazione KNX IoT per integratori e lo standard MCP, per valutare il capitolato C1. | Tutto il gruppo | 2026/10/22 | – |

## Approvazione

Il presente verbale è stato letto e approvato dal proponente.

Luogo e data

Firma